

1. The big picture

Step	File	New idea	Tool used
1	<code>step1_simple_agent.py</code>	A model + instructions = one smart reply	<code>google-generativeai</code> SDK
2	<code>step2_tool_agent.py</code>	The model can call Python functions (tools)	ADK <code>Agent</code>
3	<code>step3_multi_agent.py</code>	Several agents pass work to each other	ADK <code>SequentialAgent</code>

2. Core concepts

2.1 LLM (Large Language Model)

A program trained on huge amounts of text that predicts useful text as a reply. Gemini is an LLM. On its own it can only **produce text**. It cannot look at your files, know your students' marks, or know today's date.

2.2 Prompt and system instruction

- **Prompt / task:** what the user asks ("Write a quiz on variables").
- **System instruction (instructions):** a hidden job description that shapes every reply ("You are a teacher. Explain to Grade 9 students...").

Same model + different instructions = different behaviour. This is the cheapest way to build a specialist.

2.3 What is an agent?

A plain LLM call answers once. An **agent** is an LLM set up to *work towards a goal*. It has three parts:

AGENT = MODEL (brain) + INSTRUCTIONS (job) + TOOLS (abilities)

Agents also usually have **memory** (what happened so far) and a **loop** (think, act, look at the result, think again).

2.4 API key and `.env`

Gemini is a paid/online service, so your program proves who it is with a secret **API key**. We keep the key in a `.env` file (`GEMINI_API_KEY=...`) and load it with `load_dotenv()`. Never write the key inside the code or upload `.env` to GitHub.

2.5 Async and `await`

Calling a model means waiting for the internet. ADK is built with `async` code so waiting does not freeze the program.

- `async def main()` declares a function that can wait.
- `await something()` means "pause here until it finishes".
- `asyncio.run(main())` starts everything.

3. Step 1: The simplest agent

What the code does

1. Loads the API key and creates a `client`.
2. Defines a `SYSTEM_PROMPT` that forces a format: `REASONING:` then `FINAL OUTPUT:`.
3. Sends the task with `generate_content(...)`.
4. Splits the reply text at `FINAL OUTPUT:` to show the plan and the result separately.

Key ideas

- **Reasoning before output:** asking the model to write a short plan first often improves quality, and lets students see how it "thought".
- **Structured output by prompting:** we control the format through instructions. It works most of the time but not always, which is why the code has a fallback ("`the model did not follow the format`").
- **Limits:** no tools, no memory, no loop. It cannot check facts or act on the world.

4. Step 2: An agent with a tool

4.1 What is a tool?

A **tool is just a normal Python function** that the agent is allowed to call. The model cannot know for example a student's marks, but a function can look them up.

4.2 How does the model use a tool? (function calling)

flow is:

1. User asks: "Who scored higher, Asha or Chandra?"
2. ADK sends the question + a description of the tools to the model.
3. Model replies: "Please call `get_marks('Asha')`" <- a request, not an answer
4. ADK runs your Python function and gets `{"status": "success", "marks": 78}`
5. ADK gives that result back to the model.
6. Model asks for `get_marks('Chandra')`, gets 91.
7. Model writes the final answer: "Chandra scored higher."

This is called **function calling** (or tool use). Steps 3 to 5 repeat as many times as needed; that repetition is the agent's **loop**.

4.3 Why the docstring and type hints matter

ADK builds the tool description from:

- the **function name** (`get_marks`),
- the **docstring** (what it does, what `student_name` means),
- the **type hints** (`student_name: str, -> dict`).

The model reads these to decide *when* and *how* to call the tool. A vague docstring means the tool gets misused.

Good exercise: I rewrite the docstring badly and watch the behaviour change.

4.5 Observing the agent

`run_debug(question, verbose=True)` prints every tool call and result.

5. What Agent does

Agent (from `google.adk.agents`) is the **blueprint of one agent**. It does not run anything by itself; it *describes* the agent.

Parameter	Meaning	Example in the files
<code>name</code>	Unique identifier, used in logs and by other agents	<code>"marks_agent"</code>
<code>model</code>	Which LLM is the brain	<code>"gemini-3.5-flash-lite"</code>
<code>instruction</code>	The job description (system prompt)	<code>"You help teachers..."</code>
<code>tools</code>	List of Python functions it may call	<code>[get_marks]</code>
<code>output_key</code>	Saves the agent's final answer in shared state under this name	<code>"explanation"</code>

Other useful parameters (not used here): `description` (a one-line summary that other agents read to decide who to delegate to), `sub_agents`, `generate_content_config`.

Analogy: **Agent** is a *job contract*: name, skills, instructions, equipment. Nothing happens until someone actually gives the employee work.

6. What InMemoryRunner does

The **Runner** is the engine that actually *executes* an agent. It:

1. Receives the user's message.
2. Creates or finds a **session** (one conversation).
3. Sends the message to the agent's model.
4. Handles the tool-call loop: runs your function, returns the result to the model.
5. Passes work between agents in a multi-agent team.
6. Returns/prints the final answer.

"InMemory" means sessions and state are kept only in the computer's RAM. When the program ends, everything is forgotten. That is perfect for learning and testing. In real products

you would use a runner with a **database-backed session service** so conversations survive restarts.

`run_debug("...")` is a convenience method: it creates a session, sends one message, and prints all events. It is meant for learning and quick tests. Production apps use `run_async(...)` and handle events themselves.

7. Step 3: A multi-agent team

7.1 Why more than one agent?

One agent with a giant instruction ("explain, then design an activity, then write a quiz") becomes confusing and hard to debug. Splitting into **specialists that each do ONE job** gives:

- clearer, shorter instructions,
- easier testing (fix only the quiz writer),
- reusable parts.

This is the same idea as splitting a large program into functions.

7.2 The three specialists

Agent	Job	Reads	Saves as
<code>explainer</code>	Explains the topic to Grade 9	the user's topic	<code>explanation</code>
<code>activity_designer</code>	Designs a 15-minute activity	<code>{explanation}</code>	<code>activity</code>
<code>quiz_writer</code>	Writes a 5-question quiz	<code>{explanation}</code>	<code>quiz</code>

7.3 Shared state and `output_key`

When an agent finishes, ADK stores its answer in the session **state** (a shared dictionary) under its `output_key`:

```
state = {  
  
    "explanation": "...text from explainer...",  
  
    "activity": "...text from activity_designer...",
```

"quiz": "...text from quiz_writer..."

}

7.4 Placeholders like {explanation}

Writing {explanation} inside an instruction tells ADK: *"replace this with the value stored in state under 'explanation' before sending to the model."* That is how information travels from agent 1 to agents 2 and 3. Important: the key must already exist when that agent runs, so **order matters**.

7.5 SequentialAgent

A **workflow agent**: it does not use an LLM itself, it simply runs its **sub_agents one after another, in the listed order**. Here: explainer → activity_designer → quiz_writer. It is predictable, like an assembly line.

Other workflow agents to mention briefly:

- **ParallelAgent**: runs sub-agents at the same time (activity designer and quiz writer do not depend on each other, so they *could* run in parallel).
- **LoopAgent**: repeats sub-agents until a condition is met (for example, write, review, rewrite).

7.6 Two kinds of agents, compared

	LLM Agent (Agent)	Workflow Agent (SequentialAgent)
Decides using	the model	fixed rules you wrote
Behaviour	flexible, can vary	predictable
Use for	language, judgement, tools	controlling order and structure

8. Useful vocabulary

Term	Plain meaning
Agent	Model + instructions + tools working towards a goal
Tool	A Python function an agent may call
Function calling	The model <i>asks</i> ADK to run a tool
Runner	The engine that runs agents
Session	One conversation
State	Shared notes (a dictionary) inside a session
Event	One recorded step (a message, tool call, tool result)
<code>output_key</code>	Name under which an agent's answer is saved in state
Hallucination	Confident but made-up output
Orchestration	Deciding which agent runs, when, and in what order

9. Suggested practice questions

1. What are the three parts of an agent? Which one is missing in Step 1?
2. Why can the model not run `get_marks` by itself? Who runs it?
3. What happens if the docstring of a tool is empty?
4. In Step 3, what happens if you swap the order so `quiz_writer` runs first? (Hint: the state key is missing.)
5. What does "InMemory" mean and what is the downside?
6. Add a `get_grade(marks)` tool, then ask "Did Bikash pass?". How many tool calls do you expect?